

Measure First, Optimise Later

Benchmarking in C#

Marcel Barlik

April 28, 2026

Agenda

- ▶ **Context & Motivation** — Why performance measurement matters
- ▶ **What is Benchmarking?** — Definitions and what we can measure
- ▶ **Key Benefits** — What you gain by benchmarking
- ▶ **BenchmarkDotNet** — The standard tool for C#
- ▶ **Key Takeaways** — What to do next

Expected time: 10-15 minutes

The Performance Problem

Symptoms we hear

- ▶ “This function feels slow”
- ▶ “P99 latencies are out of control”
- ▶ “We are spending a lot on cloud compute”
- ▶ Users complain about long app loading times

Why they're hard to fix

- ▶ “Slow” is not actionable
- ▶ We don't know *where* the time goes
- ▶ Fixes are guesswork without data
- ▶ Regressions go unnoticed until production

The core problem

You cannot optimize what you have not measured.

The Chain Is Only as Strong as Its Weakest Link

A service can only be as fast as the slowest dependency it relies on.

Common bottleneck dependencies

- ▶ Database queries and round-trips
- ▶ External API calls
- ▶ Message queues and event buses
- ▶ Cache misses (falling back to slower sources)
- ▶ Network I/O and serialisation

The analogy: “A Ferrari on a pebble road drives at pebble-road speed.”

Key Insight

Identify your actual bottlenecks before optimising. Often it's not the code – it's what the code is waiting for.

Intuition vs. Reality

What developers assume is slow

- ▶ String operations
- ▶ Simple loops
- ▶ Math calculations
- ▶ Reading from memory

What profilers actually find

- ▶ Database round-trips
- ▶ Network and I/O calls
- ▶ Serialisation overhead
- ▶ Lock contention and GC pressure

Donald Knuth, 1974

“We should forget about small efficiencies, say about 97% of the time: premature optimization is the root of all evil. Yet we should not pass up our opportunities in that critical 3%.”

What is Benchmarking?

Definition

Benchmarking is the practice of **measuring the performance of a specific piece of code** under controlled, repeatable conditions – producing data you can compare, track, and act on.

What makes a good benchmark?

- ▶ **Isolated** – tests one thing at a time
- ▶ **Repeatable** – same code produces comparable results
- ▶ **Statistically sound** – accounts for warmup, JIT, and noise
- ▶ **Meaningful** – measures something that reflects real usage

Micro vs. Macro Benchmarks

Micro Benchmarks

- ▶ Measure a single method
- ▶ Fast to run, easy to iterate
- ▶ Tool: BenchmarkDotNet

Best for: comparing implementations, validating a specific optimisation, tracking regressions at the unit level

Both are valuable – they answer different questions. Start with micro when optimising a specific hot path.

Macro Benchmarks

- ▶ Measure end-to-end behaviour
- ▶ Closer to real-world load
- ▶ Tools: load testing, profilers, APM

Best for: finding real bottlenecks in production scenarios, validating system-level performance goals

What Can We Measure?

Time

- ▶ Mean execution time (ns / μ s / ms)
- ▶ Min, max, percentiles
- ▶ Statistical error and variance

Memory

- ▶ Heap allocations per operation
- ▶ GC collection frequency
- ▶ Peak working set

Throughput

- ▶ Operations per second
- ▶ Requests per second
- ▶ Bytes processed per second

Hardware counters

- ▶ CPU cache misses
- ▶ Branch mispredictions
- ▶ CPU cycles and instructions

Without vs. With Benchmarks

Without Benchmarks

- ▶ Optimise based on gut feeling
- ▶ Cannot prove a fix actually helped
- ▶ Regressions slip into production unnoticed
- ▶ Performance reviews become debates

With Benchmarks

- ▶ Optimise based on evidence
- ▶ Prove improvements with numbers
- ▶ Catch regressions in CI before they ship
- ▶ Performance reviews become conversations

Key Benefits of Benchmarking

Data-driven Decisions

Optimise the code that is *actually* slow, not the code you assume is slow.

Catch Regressions Early

Benchmarks can run in CI. A performance regression often indicates a bug or oversight.

Validate Improvements

Quantifiable wins and side-by-side comparisons.

Compare Alternatives Confidently

Choosing between two implementations? The numbers help decide.

Where Benchmarking Fits in the Dev Lifecycle

1. **Development** – Benchmark a hot path while writing or refactoring it. Fail fast, iterate quickly.
2. **Code Review** – Attach benchmark results to the pull request as evidence. Performance becomes part of the review.
3. **CI / CD** – Run a benchmark suite automatically. Gate merges on regressions beyond an agreed threshold.
4. **Production** – Profile real workloads to discover the next bottleneck and feed that back into step 1.

Key insight: Performance is a feature – treat it like one throughout the whole cycle.

The standard micro-benchmarking library for .NET

What it handles for you

- ▶ JIT warmup and tiered compilation
- ▶ Multiple iterations and statistical analysis
- ▶ Outlier detection and error reporting
- ▶ Memory allocation tracking ([MemoryDiagnoser])
- ▶ Baseline comparisons and ratios
- ▶ Export to Markdown, HTML, CSV

Get started in one command:

```
dotnet add package BenchmarkDotNet
```

Credibility

- ▶ Used by the .NET runtime team
- ▶ Used by Microsoft ASP.NET team
- ▶ Industry standard across the .NET ecosystem

A Concrete C# Example

Comparing string concatenation approaches

```
1  [MemoryDiagnoser]
2  public class StringBenchmarks
3  {
4      [Params(10, 100, 1000)]
5      public int N;
6
7      [Benchmark(Baseline = true)]
8      public string Concatenation()
9      {
10         var s = "";
11         for (var i = 0; i < N; i++) s += i;
12         return s;
13     }
14
15     [Benchmark]
16     public string UseStringBuilder()
17     {
18         var sb = new StringBuilder();
19         for (var i = 0; i < N; i++) sb.Append(i);
20         return sb.ToString();
21     }
22 }
```

Reading the Results

Real results – N = 1000 (full results in appendix)

Method	Mean	Error	StdDev	Ratio	Gen0	Allocated
Concatenation	56.45 μ s	0.31 μ s	0.24 μ s	1.00	56.58	2,774 KB
UseStringBuilder	2.16 μ s	0.01 μ s	0.01 μ s	0.04	0.29	14.4 KB

- ▶ **Mean** – average execution time per operation
- ▶ **Error / StdDev** – statistical noise in the measurement
- ▶ **Ratio** – relative to the baseline (1.00)
- ▶ **Gen0** – GC collections per 1,000 operations
- ▶ **Allocated** – heap memory per operation

Result: StringBuilder is **25 $\times$** faster, triggers **195 $\times$** fewer GC collections, and allocates **192 $\times$** less memory.

BenchmarkDotNet: Do's and Don'ts

Do's

- ▶ Use `[MemoryDiagnoser]` to track allocations
- ▶ Use `[Params(...)]` to test multiple scenarios
- ▶ Compare apples-to-apples – same inputs, same environment
- ▶ When running on UNIX-like systems, use `nice -n -19` to reduce noise from other processes

Don'ts

- ▶ Don't benchmark in Debug mode
- ▶ Don't include I/O or network calls in micro-benchmarks
- ▶ Don't optimize without data

Key Takeaways

1. **You cannot optimise what you have not measured** – vague complaints are not actionable; numbers are
2. **Developer intuition about bottlenecks is often wrong** – let a profiler or benchmark find the real hot path
3. **Benchmarking turns assumptions into evidence** – before/after numbers end debates and build confidence
4. **BenchmarkDotNet is the standard tool for .NET** – it handles the hard parts (warmup, statistics, allocations)
5. **Integrate early** – benchmarks in CI catch regressions before they reach production

Next step: Look through your most hit service, and find one hot path to benchmark and optimise.

Questions?

What performance problems are you dealing with right now?

Have you already used BenchmarkDotNet? What did you learn?

References & Further Reading I

- ▶ **BenchmarkDotNet** – <https://benchmarkdotnet.org> – official docs, guides, and configuration reference
- ▶ **BenchmarkDotNet GitHub** – <https://github.com/dotnet/BenchmarkDotNet> – source, issues, and real-world usage examples from the .NET ecosystem
- ▶ **.NET Blog: Performance Improvements** – <https://devblogs.microsoft.com/dotnet/> – the .NET team publishes benchmark-driven perf posts each release
- ▶ **.NET Blog: .NET 10 Improvements** – <https://devblogs.microsoft.com/dotnet/performance-improvements-in-net-10/> – detailed benchmarks showing the impact of .NET 10 optimisations

Benchmark Environment

```
BenchmarkDotNet v0.15.8, Linux NixOS 25.11 (Xantusia)
AMD Ryzen 7 9850X3D 5.61GHz, 1 CPU, 16 logical and 8 physical cores
.NET SDK 10.0.201
[Host]      : .NET 10.0.5 (10.0.5, 10.0.526.15411), X64 RyuJIT x86-64-v4
DefaultJob : .NET 10.0.5 (10.0.5, 10.0.526.15411), X64 RyuJIT x86-64-v4
```

Method	N	Mean	Error	StdDev	Ratio	RatioSD	Gen0	Gen1	Allocated	Alloc Ratio
Concatenation	10	53.37 ns	0.690 ns	0.646 ns	1.00	0.02	0.0067	–	336 B	1.00
UseStringBuilder	10	24.01 ns	0.180 ns	0.159 ns	0.45	0.01	0.0030	–	152 B	0.45
Concatenation	100	948.49 ns	5.435 ns	4.818 ns	1.00	0.01	0.4139	–	20856 B	1.00
UseStringBuilder	100	228.48 ns	3.308 ns	3.094 ns	0.24	0.00	0.0253	–	1280 B	0.06
Concatenation	1000	56452.99 ns	310.789 ns	242.644 ns	1.00	0.01	56.5796	0.7935	2840456 B	1.000
UseStringBuilder	1000	2163.83 ns	12.941 ns	10.104 ns	0.04	0.00	0.2899	0.0076	14712 B	0.005